

Declarative Automation Bundles

databricks.yml, targets and deployment modes, variables and overrides, and the CLI lifecycle.

A bundle versions **the project**, not just the code. The jobs, the pipelines, the schedules and the configuration all live in the repository beside the notebooks.

What a bundle contains

PART	WHAT IT HOLDS
<code>databricks.yml</code>	The bundle definition: name, resources, targets, variables
Source files	Notebooks, Python files, SQL — the code itself
Resource definitions	Jobs, pipelines and other workspace assets, as YAML
Targets	dev, staging, prod — each with its own workspace and overrides
Variables	Values that differ per target

ON THE EXAM

All of it lives in one repository and goes through the same review as the code. That is the point.

The main top-level mappings

MAPPING	PURPOSE
<code>bundle</code>	The bundle's name, and its deployment metadata
<code>include</code>	Other YAML files to pull in, so one file does not hold everything
<code>variables</code>	Declared variables, with optional defaults
<code>resources</code>	Jobs, pipelines and other assets to deploy
<code>targets</code>	The environments, each able to override anything above
<code>workspace</code>	Paths in the target workspace, such as <code>root_path</code>
<code>artifacts</code>	Things to build during deployment, such as a Python wheel
<code>run_as</code>	The identity deployed resources run as

ON THE EXAM

Everything above targets can be overridden inside a target. That single rule is what makes one file serve three environments.

A target is an environment. Same code, same resource definitions, **different values**. That is the whole mechanism for promoting across dev, test and prod.

Development mode

- ◆ **Set mode: development** on the target, and a set of default behaviours applies.
- ◆ **Resource names are prefixed** with a dev marker including your username, so two developers do not collide.
- ◆ **Jobs and pipelines are tagged** with a dev tag.
- ◆ **Pipelines are marked development: true.**
- ◆ **--compute-id becomes available** on deploy, overriding every cluster definition in the bundle.

ON THE EXAM

The prefixing is why several engineers can deploy the same bundle to one workspace without overwriting each other's jobs.

Production mode

- ◆ **Set mode: production** and you get validations rather than conveniences.
- ◆ **Validates that all deployed pipelines are marked development: false.**
- ◆ **Validates that the current Git branch matches the branch specified in the target**, if one is specified.
- ◆ **That branch check can be overridden with `--force`**, which is occasionally necessary and usually a smell.

ON THE EXAM

Production mode's job is to stop you deploying something you did not mean to, from a branch you did not mean to deploy from.

Production without a service principal

- ◆ Databricks recommends service principals for production deployments, enforced by setting `run_as`.
- ◆ If you do not use one, extra validations apply.
- ◆ It validates that `artifact_path`, `file_path`, `root_path` and `state_path` are not overridden to a specific user.
- ◆ **The reasoning:** production assets deployed under one person's path belong to that person, and vanish with them.

ON THE EXAM

This is module 19's `run-as` argument again, enforced by the tooling rather than left to good intentions.

A variable is declared once, given a default, and **overridden per target**. That is how one catalog name serves three environments.

Declaring and overriding

WHERE	WHAT IT LOOKS LIKE
Declare	<code>variables: catalog: {default: dev_catalog}</code>
Reference	<code>catalog: \${var.catalog}</code>
Override in a target	<code>targets: prod: variables: {catalog: prod_catalog}</code>
Override at the command line	<code>databricks bundle deploy -t prod --var="catalog=other"</code>

ON THE EXAM

Four places the same value can come from. Command line beats target, target beats default.

Substitution syntax

- ◆ `${var.name}` references a declared variable.
- ◆ `${bundle.target}` gives the name of the target being deployed.
- ◆ `${workspace.current_user.userName}` is who is deploying, which is how dev mode builds its prefix.
- ◆ `${resources.jobs.my_job.id}` references another resource, so one asset can point at another.

ON THE EXAM

Referencing resources by name rather than by id is what lets the same definition deploy to three workspaces where the ids all differ.

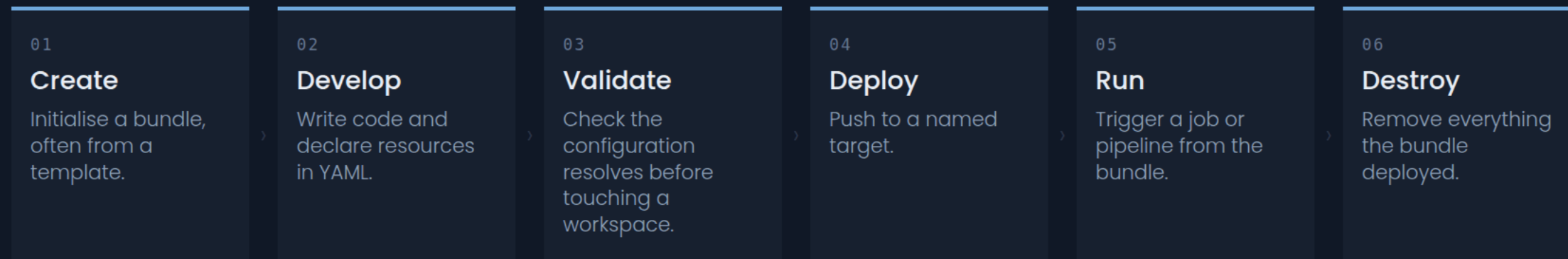
One thing variables cannot do

- ◆ Variables cannot dynamically set include directives.
- ◆ So you cannot include a different set of YAML files per target using a variable.
- ◆ The workaround is to substitute in CI, editing databricks.yml before running the deploy.
- ◆ In practice, prefer overrides. Define everything and override what differs, rather than including different files.

ON THE EXAM

If a design needs entirely different resources per environment, that is usually a sign the environments have drifted apart.

The lifecycle



DATABRICKS' OWN SEQUENCE

Six stages, and this is Databricks' own framing of the lifecycle. Validate sits deliberately before deploy.

The CLI commands

COMMAND	WHAT IT DOES
<code>databricks bundle init</code>	Create a new bundle, optionally from a template
<code>databricks bundle validate</code>	Check the configuration resolves. No workspace changes.
<code>databricks bundle deploy -t prod</code>	Deploy the named target
<code>databricks bundle run my_job</code>	Trigger a job or pipeline defined in the bundle
<code>databricks bundle summary</code>	Show what is deployed for a target
<code>databricks bundle destroy</code>	Remove everything the bundle deployed

ON THE EXAM

The `-t` flag names the target. Omit it and you get the default target, which is how accidents happen.

What bundles can deploy

- ◆ **Lakeflow Jobs**, with all their tasks, schedules and settings.
- ◆ **Lakeflow Spark Declarative Pipelines**, including ingestion pipelines from Lakeflow Connect.
- ◆ **Other workspace assets**, and the set continues to grow — alerts became definable in bundles recently.
- ◆ **Artifacts built during deployment**, such as a Python wheel compiled from your source.

ON THE EXAM

Module 06 noted this from the other side: the Lakeflow Connect docs name bundles as the way to deploy ingestion pipelines.

Bundles and drift

- ◆ **Editing a bundle-deployed job in the workspace UI creates drift** between the workspace and the repository.
- ◆ **The next deploy overwrites the manual change**, usually without warning anybody.
- ◆ **The repository is the source of truth.** Fix the YAML, review it, deploy it.
- ◆ **Restrict CAN MANAGE on production jobs**, so the tempting edit is not available in the first place.

ON THE EXAM

An urgent fix made in the UI at 2am is invisible to everyone and disappears at the next deploy. That is worse than the original outage.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Different catalog per environment	A variable, overridden in each target
Two developers overwriting each other's jobs	mode: development, which prefixes resource names
Deploying to prod from the wrong branch	Production mode's Git branch validation
A pipeline reached prod still in development mode	Production mode validates development: false
Check the configuration without deploying	databricks bundle validate
A manual UI fix disappeared	Drift. The next deploy overwrote it.
Production deployed under one person's path	Use run_as with a service principal

ON THE EXAM

Seven lines. The second and third are the two deployment-mode behaviours most likely to be tested directly.

Four engineers share a development workspace and each deploys the same bundle from their own branch to test changes. Their target is configured with mode: production, which they chose because it sounded more rigorous.

They keep overwriting each other's jobs. Whoever deployed most recently has their version of every job in the workspace.

What should they change?

- ☐ A Give each engineer their own workspace
- ☐ B Set mode: development on that target, which prefixes resource names per user
- ☐ C Add a variable for the job name and have each engineer override it
- ☐ D Have only one engineer deploy, and the others request changes from them

◆ PAUSE AND DECIDE

What should they change?

A A workspace each

Would work, at considerable cost and administration. The feature that solves this without any of that already exists.

C A variable for the job name

Reimplements development mode by hand, and depends on every engineer remembering to set it correctly every time.

B mode: development

Correct. Development mode prefixes resource names with a dev marker including the deploying user, so four engineers get four sets of jobs.

D One person deploys for everyone

Solves collisions by removing the ability to test independently, which is the thing a development environment is for.

THE SIGNAL

They chose production mode because it sounded rigorous. Modes are not a quality setting; they select behaviours suited to a purpose.

A release engineer runs `databricks bundle deploy -t prod`. It fails, reporting that the current Git branch does not match the branch specified in the target. The target specifies `main`; she is on a feature branch.

She is under time pressure and a colleague suggests adding `--force` to get the deployment through.

What is happening, and what should she do?

- ☐ A A bug in the CLI; `--force` is the documented workaround
- ☐ B Production mode validates the Git branch. `--force` would bypass it, but the correct action is to deploy from `main`.
- ☐ C The bundle is missing a git property, which must be added to the target
- ☐ D Her Git credentials have expired, and the CLI cannot read the branch

◆ PAUSE AND DECIDE

What is happening, and what should she do?

- A

A bug; --force is the workaround

Documented behaviour, not a bug. --force does override it, and calling a safety check a bug is how safety checks get removed.
- C

A missing git property

Backwards. The git property is clearly present, which is why the target knows to expect main.
- B

Production mode's branch validation; deploy from main

Correct. The check exists to stop exactly this: production deployed from a branch that has not been through review.
- D

Expired credentials

The error names a branch mismatch, not an authentication failure. The CLI read the branch successfully to compare it.

THE SIGNAL

--force exists and works. The question is whether the check was wrong, or whether you were. Here, the check was right.

A bundle deploys one job to three targets. In dev it should write to dev_catalog on a 2-worker cluster with no schedule; in prod, to prod_catalog on an 8-worker cluster running nightly at 02:00.

An engineer proposes maintaining three copies of the job YAML, one per environment, so each is explicit and easy to read.

What is the better approach?

- ☐ A Three copies is clearer and avoids indirection; keep the proposal
- ☐ B One job definition using variables, with each target overriding catalog, worker count and schedule
- ☐ C One definition, with the notebook reading an environment variable and branching internally
- ☐ D Three separate bundles, one per environment

◆ PAUSE AND DECIDE

What is the better approach?

- A

 Three copies of the YAML
Readable on day one and divergent by month three. A change to the job must be made three times, and one will be missed.
- B

 One definition, variables overridden per target
Correct. This is exactly what the objective describes: environment-specific configuration through variables and overrides, one codebase across targets.
- C

 Branching inside the notebook
Moves configuration into code, where it cannot be reviewed as configuration. It also cannot set cluster size or schedule, which are outside the notebook.
- D

 Three bundles
The same divergence problem as A, with more machinery. The code is identical; only the configuration differs.

THE SIGNAL

Option C fails on a hard limit as well as a soft one: a notebook cannot set its own cluster size or its job's schedule.

During an overnight incident, an on-call engineer edits a bundle-deployed production job in the workspace UI, increasing a timeout and changing a cluster node type. The job recovers and the incident is closed.

Three days later, a routine bundle deployment runs. The following night the same failure recurs.

What happened, and what is the durable fix?

- ☐ A The deployment reverted the manual change; make the fix in the bundle YAML, review it and deploy
- ☐ B The deployment failed and left the job in a broken state
- ☐ C The bundle should be redeployed with `--force` to preserve manual edits
- ☐ D The engineer's change was never saved

◆ PAUSE AND DECIDE

What happened, and what is the durable fix?

- A

 The deploy reverted it; fix the YAML instead
Correct. The bundle is the source of truth. A deployment reconciles the workspace to the YAML, which still had the old timeout.
- B

 The deployment failed
It succeeded. The job ran; it simply ran with the configuration the bundle specified, which was the pre-incident one.
- C

 Redeploy with `--force` to preserve edits
Inverts what `--force` does. It overrides validations; it does not protect manual workspace changes from being overwritten.
- D

 The change was never saved
It was saved and it worked, which is why the incident closed. It was overwritten later, which is a different thing.

THE SIGNAL

The fix worked and was invisible. Nothing in the repository recorded it, so nothing in the repository preserved it.

One definition, many targets. **Variables carry what differs, modes carry what should be enforced, and the repository is the source of truth.**